

Snake Game Lab 4 - Chat Conversation

Pair-Programming Transcript | Software Engineering Lab

User Prompt #1

Read the readme. Can you implement task 1 and also add wasd key controls.

Assistant Response

Implementation Summary

1. Task 1: Refine Collision Detection

Updated `game/snake.py` to resolve instant self-collision bugs caused by rapid or opposite direction key presses:

- **180-Degree Reversal Protection:** Prevented the snake from immediately turning 180° into the body segment directly behind the head (e.g., pressing `Left / A` while moving `Right / D`).
- **Direction Buffer Queue:** Implemented a pending turn queue (`next_directions` up to 2 turns) so rapid consecutive key presses within a single movement tick are executed cleanly in sequence rather than causing unfair collisions.

2. WASD Key Controls

- Verified WASD input support (`pygame.K_w`, `pygame.K_a`, `pygame.K_s`, `pygame.K_d`) alongside standard arrow keys in `game/game_engine.py`. Both input schemes work seamlessly for controlling the snake.

Verification

Ran test suites covering:

1. Prevention of instant 180° turns into body segments.
2. Safe execution of rapid multi-key input sequences within a tick.
3. Proper direction updates using WASD keys.

All tests passed successfully. You can launch the game via:

```
python main.py
```

User Prompt #2

Can you implement task 2 now.

Assistant Response

Task 2 Implementation Summary

Implemented the **Game Over Condition** screen and input handling:

1. **Game Over Overlay:**

- Added a translucent dark overlay rendered over the game screen when the snake hits a wall or collides with itself.
- Displayed centered **"GAME OVER"** title in bold red typography.
- Displayed **"Final Score: X"** prominently on screen.
- Added user prompt: **"Press any key to exit"**.

2. Graceful Input Handling:

- Modified `handle_keydown` in `game/game_engine.py` to set a `should_quit` flag when any key is pressed during the Game Over screen.
- Updated the main event loop in `main.py` to check `should_quit` and gracefully exit the application.

Verification

Created unit tests verifying:

- Game Over screen rendering with final score.
- Collision trigger state transitions.
- Key press input capturing on the Game Over screen.

All tests ran and passed cleanly.

User Prompt #3

can we do task 3. Also add a high score value that shows up with the game over screen. It should last only for that session till the player quits.

Assistant Response

Task 3 & Session High Score Implementation Summary

1. Session High Score

- Added `high_score` tracking in `game/game_engine.py`.
- High score automatically updates whenever the player score increases or upon Game Over.
- Displayed in the top right during gameplay (**High Score: X** in Gold) and prominently on the **Game Over** screen (**Session High Score: X**).
- High score persists across restarts within the active application session and resets when quitting.

2. Replay Option with Difficulty Selection

Implemented `restart()` in `GameEngine` and updated the Game Over keyboard menu:

- `[1]` / `[E]`: Restart on **Easy** speed (6 moves/sec)
- `[2]` / `[M]`: Restart on **Medium** speed (10 moves/sec)
- `[3]` / `[H]`: Restart on **Hard** speed (16 moves/sec)
- `[SPACE]` / `[R]`: Restart at current difficulty
- `[ESC]` / `[Q]`: Quit game gracefully

Verification

Ran comprehensive unit tests verifying:

- Persistence of `high_score` across game restarts in the session.
- Difficulty speed setting updates (Easy, Medium, Hard).
- Graceful quitting via `ESC` or `Q`.

All unit tests passed.

User Prompt #4

On the game over screen, press space to restart at current difficulty is not correctly mentioned. Clearly mention which key to press for which option in a more organised manner. Also implement task 4.

Assistant Response

Summary of Enhancements

1. Task 4: Sound Feedback Implementation

Created `game/sound.py` to generate synthetic audio effects directly in memory (zero external file dependencies required):

- **Food Eat Sound:** High-pitched 2-tone arpeggio played when the snake consumes food.
- **Game Over Sound:** Arcade-style descending pitch sound played when the snake collides with a wall or itself.

2. Re-organized Game Over Screen UI

Updated `game/game_engine.py` to format all key options on the Game Over screen:

```

    GAME OVER

    Final Score: 12
    Session High Score: 15

    [ SPACE ] : Play Again (Current Speed)
    [ 1 ] Easy   |   [ 2 ] Medium   |   [ 3 ] Hard
    [ ESC / Q ] : Quit Game
```

Complete Task Checklist Summary

- [x] **Task 1:** Refined collision accuracy, guarded against 180° instant reversals into the neck, and added input queue buffering.
- [x] **WASD Controls:** Added full `W`, `A`, `S`, `D` support alongside standard arrow keys.
- [x] **Task 2:** Game Over screen with semi-transparent overlay and score display.
- [x] **Task 3:** Replay menu allowing difficulty selection (Easy: `1`, Medium: `2`, Hard: `3`, Current Speed: `SPACE`).
- [x] **Session High Score:** Displayed live during gameplay and on the Game Over screen.
- [x] **Task 4:** Sound feedback for eating food and Game Over moments.

Run the game using:

```
python main.py
```
